

Autonomous Navigation on the Robile Platform: Path Planning, Localization, and Exploration

Marco Atef Cosman Gadallah
Autonomous Systems, M.Sc.
Hochschule Bonn-Rhein-Sieg
Sankt Augustin, Germany
marco.gadallah@smail.inf.h-brs.de

Abstract—This report presents the development and integration of autonomous navigation functionalities on the KUKA Robile omnidirectional mobile platform. The system comprises three main components: (i) a two-layer path and motion planning system combining an A* global planner with a potential field local planner, (ii) a Monte Carlo Localization particle filter for robot pose estimation within a known map, and (iii) a frontier-based autonomous exploration system integrated with SLAM Toolbox for incremental map building. All components are implemented as ROS2 nodes within a single Python package, tested in Gazebo simulation, and designed for deployment on the real Robile hardware. The complete implementation is available at <https://github.com/HBRS-AMR/amr-ws25-projects-amr-ws26-perception/tree/main>.

I. INTRODUCTION

Autonomous mobile robot navigation requires the integration of several core capabilities: the ability to plan collision-free paths, to localize within an environment, and to explore unknown spaces. This project addresses all three aspects on the Robile platform, a KUKA omnidirectional mobile base equipped with a 2D LiDAR scanner and wheel encoders.

The project is structured in three parts that build upon each other. First, a path and motion planning pipeline enables the robot to navigate to arbitrary goals on a known map. Second, a particle filter provides localization by correcting odometry drift using laser scan measurements. Third, a frontier-based exploration module autonomously selects navigation goals to incrementally map an unknown environment using SLAM.

All components are implemented in Python as ROS2 Humble nodes within a single `ament_python` package called `amr_perception`. The system was developed and validated entirely in Gazebo simulation using the `closed_walls_obstacles` world environment before deployment on the physical Robile.

II. ENVIRONMENT SETUP AND MAP BUILDING

Before implementing the planning and localization components, a map of the simulation environment was created using SLAM Toolbox [1]. The process involved launching the Robile in Gazebo, running `slam_toolbox` in online asynchronous mode, and teleoperating the robot through the environment using `teleop_twist_keyboard`. The resulting occupancy grid map was saved using the Nav2 `map_saver_cli` utility,

producing a 159×145 pixel grid at 0.05 m resolution, representing an approximately $8 \text{ m} \times 7 \text{ m}$ indoor environment.

This static map serves as input for the A* planner and the particle filter in Parts 1 and 2. For Part 3 (exploration), no pre-built map is used; instead, SLAM Toolbox runs continuously alongside the exploration node.

A shared `MapUtils` module provides coordinate conversion between world coordinates (meters) and grid indices, cell occupancy queries, neighbor lookups, and obstacle inflation for all nodes that interact with the occupancy grid.

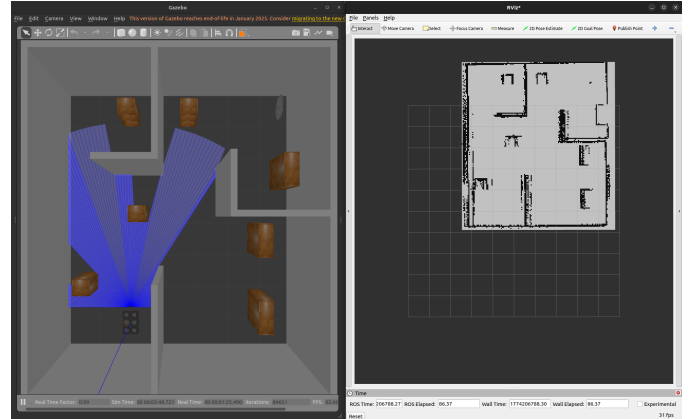


Fig. 1. The map of closed walls world with rooms and obstacles.

III. PATH AND MOTION PLANNING

A. Architecture

The planning system follows a two-layer architecture. A global planner (A*) computes a coarse path on the occupancy grid, which is then simplified into a set of waypoints. A local planner (potential field) navigates the robot between consecutive waypoints while reactively avoiding obstacles. A coordinator node manages the sequencing of waypoints.

B. A* Global Planner

The A* planner operates on an inflated version of the occupancy grid, where obstacles are expanded by a configurable radius (7 cells = 0.35 m) to ensure planned paths maintain a safety margin from walls. The algorithm uses

8-connected grid neighbors with Euclidean distance as the heuristic. The robot's current position is obtained via TF lookup (`map→base_footprint`) rather than raw odometry, ensuring correct positioning when the particle filter is active.

C. Waypoint Extraction

The raw A* path typically contains hundreds of grid cells. To produce a manageable set of intermediate goals for the local planner, the Ramer-Douglas-Peucker (RDP) line simplification algorithm is applied. RDP identifies points where the path changes direction significantly and discards redundant points along straight segments. A configurable tolerance parameter ($\epsilon = 0.3$ m) controls the level of simplification. After RDP, a maximum spacing constraint (1.5 m) ensures that no two consecutive waypoints are too far apart by inserting interpolated points where necessary.

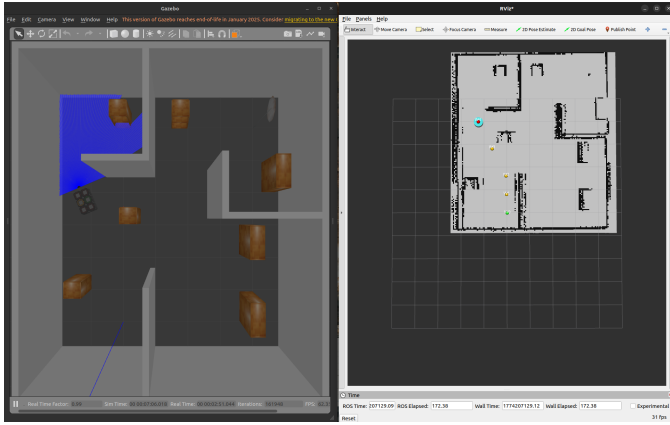


Fig. 2. Example of a planned path with extracted waypoints.

D. Potential Field Local Planner

The potential field planner computes velocity commands by combining attractive and repulsive forces. The attractive force is a unit vector pointing from the robot toward the current waypoint, scaled by a gain $k_a = 1.5$. The repulsive force is computed from the laser scan: for each detected obstacle within a configurable influence distance $\rho_0 = 0.5$ m, a repulsive force proportional to $k_r(1/d - 1/\rho_0)/d^2$ is applied, where d is the distance to the obstacle.

Since the Robile is omnidirectional, the total force vector in the map frame is transformed to the robot's local frame using the inverse rotation matrix:

$$\begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix} \begin{bmatrix} F_x \\ F_y \end{bmatrix} \quad (1)$$

where θ is the robot's heading and (F_x, F_y) is the total force in the map frame.

An exponential moving average (EMA) filter smooths velocity commands to prevent oscillatory motion, particularly near obstacles. The control loop runs at 10 Hz.

E. Planner Coordinator

The coordinator node receives navigation goals (from RViz or the exploration module), forwards them to the A* planner, and sequentially publishes waypoints to the potential field planner via `/waypoint_goal`. It monitors progress using TF-based distance checks and implements stuck detection: if no progress is made for a configurable timeout (10 s), the current waypoint is skipped or the path is replanned.

IV. MONTE CARLO LOCALIZATION

A. Algorithm Overview

[2] The particle filter maintains a set of $N = 1500$ weighted particles, each representing a hypothesis for the robot's pose (x, y, θ) . The algorithm follows the standard predict-update-resample loop triggered by odometry changes.

B. Initialization

Particles are initialized using the robot's initial odometry as a hint: 50% of particles are distributed with a Gaussian spread (1.0 m standard deviation) around the odometry starting position, with orientations concentrated near the actual heading. The remaining particles are distributed uniformly across free space for global localization robustness.

C. Motion Model

The odometry motion model decomposes each movement into a rotate-translate-rotate sequence. Gaussian noise is added proportionally to the magnitude of each component, with parameters $\alpha_1 = \alpha_3 = 0.05$ and $\alpha_2 = \alpha_4 = 0.02$. Particles that move into occupied cells are rejected entirely (position and orientation unchanged) to maintain consistency.

D. Sensor Model

For each particle, the expected laser scan is simulated via ray casting on the occupancy grid. The ray caster steps through grid cells along each beam direction until hitting an occupied cell or reaching maximum range. To balance accuracy and computational cost, only 50 subsampled beams are used per update.

The weight of each particle is computed using a beam-based likelihood model combining a Gaussian hit component ($z_{\text{hit}} = 0.9$, $\sigma = 0.15$ m) and a uniform random component ($z_{\text{rand}} = 0.1$). Weights are accumulated in log-space to prevent numerical underflow, then normalized.

During the first 30 updates (convergence phase), a tighter σ ($0.7 \times$ nominal) and forced resampling accelerate convergence.

E. Resampling

Low-variance systematic resampling is used, with adaptive random particle injection: when particle variance is high (localizing), 3% of particles are replaced with random poses; when converged (low variance), injection drops to 1%.

F. TF Broadcasting

The particle filter publishes the estimated $\text{map} \rightarrow \text{odom}$ transform, computed as:

$$\mathbf{t} = \mathbf{p}_{\text{map}} - R(\Delta\theta) \cdot \mathbf{p}_{\text{odom}} \quad (2)$$

where $\Delta\theta = \hat{\theta} - \theta_{\text{odom}}$ and R is the 2D rotation matrix. This replaces the static identity transform used during planning-only testing and enables all other nodes to operate correctly in the map frame.

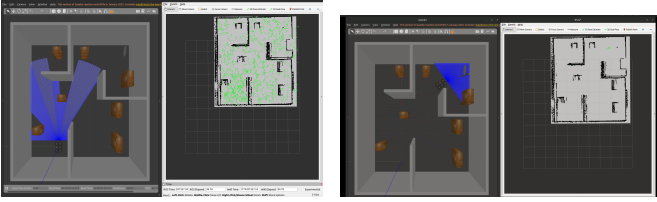


Fig. 3. Left: Particle filter initialized. Right: Particle filter convergence.

V. FRONTIER-BASED EXPLORATION

A. Overview

[3], [4] The exploration module enables fully autonomous mapping of an unknown environment. It operates alongside SLAM Toolbox, which publishes a continuously evolving occupancy grid on `/map`. The exploration node detects frontiers on this map, selects the most promising one, and sends it as a navigation goal to the existing planning pipeline.

B. Frontier Detection

A frontier cell is defined as a free cell (occupancy value < 25) that is adjacent (4-connected) to at least one unknown cell (value $= -1$). The detection scans the entire map grid every 3 seconds.

C. Frontier Clustering

Detected frontier cells are grouped into contiguous clusters using BFS flood fill with 8-connectivity. Clusters smaller than a minimum size threshold (5 cells) are discarded as insignificant.

D. Goal Selection

Each cluster is scored using a weighted combination of size (information gain) and distance (travel cost):

$$\text{score} = w_s \cdot \frac{|C|}{|C|_{\text{max}}} - w_d \cdot \frac{d}{d_{\text{max}}} \quad (3)$$

where $|C|$ is the cluster size, d is the Euclidean distance from the robot, and $w_s = 2.0$, $w_d = 1.0$ are configurable weights. Larger frontiers are preferred as they likely reveal more new area.

E. Blacklisting and Timeout

Frontiers that the robot fails to reach after 3 attempts or that exceed a 30-second navigation timeout are blacklisted. The exploration terminates when no valid frontiers remain.

F. Integration

The frontier explorer publishes goals on `/exploration_goal`, which the planner coordinator already subscribes to. This allows the entire planning pipeline ($A^* \rightarrow \text{waypoints} \rightarrow \text{potential field}$) to be reused without modification. SLAM Toolbox handles the $\text{map} \rightarrow \text{odom}$ transform during exploration, eliminating the need for the particle filter in this mode.

VI. RESULTS AND DISCUSSION

The complete system was validated in Gazebo simulation using the `closed_walls_obstacles` world. The planning pipeline successfully navigates between rooms through doorways, with A^* routing around obstacles and the potential field planner handling local adjustments. The particle filter converges within approximately 10–15 seconds of robot motion, after which the localization-corrected planning operates reliably. The exploration module autonomously maps the environment by visiting frontier regions in order of decreasing information gain.

Key parameter tuning insights include: (i) the attractive-to-repulsive force ratio ($k_a/k_r \approx 5$) should strongly favor attraction when A^* already ensures obstacle-free routing; (ii) the obstacle inflation radius must account for the robot's physical width; (iii) the particle filter benefits significantly from odometry-hinted initialization and a convergence boost phase.

To check results.

REFERENCES

- [1] S. Macenski and I. Jambrecic, "SLAM Toolbox: SLAM for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.
- [2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [3] O. Khatib, "Real-time obstacle avoidance for manipulators and mobile robots," *Int. J. Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.
- [4] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE Int. Symp. Computational Intelligence in Robotics and Automation*, 1997, pp. 146–151.